

Governance Drift: Measuring Divergence Between Approved Intent and Operational Reality in Cloud-Native Systems

Obede Bessa

Independent Researcher — obedebessa@gmail.com

Abstract—Configuration drift is a two-place, memoryless comparison of desired and observed configuration. We define *governance drift* as a three-place, history-anchored relation: operational reality diverges from its current governance context and the immutable basis under which it was approved, or the evidence needed to decide that relation is lost. A resource can match Git while violating current policy, relying on expired authority, or running unapproved content. We contribute a five-component governance-divergence vector plus an epistemic mask; an append-only approval-snapshot model; separation and tier-minimality results; and a cumulative detection architecture. In a twelve-scenario closed world, complete-set scoring realizes the predicted staircase; an independent union of plane-local signals exactly diagnoses 9/12 scenario sets versus 12/12 for the admitted-basis join. In a pinned Kind/Flux/Kyverno laboratory, nine scenarios were each repeated 20 times at randomized-phase 0.5-, 2-, and 10-second cadences. The evaluator detected 538/540 observations with 100% conditional exact-set accuracy; a cluster bootstrap over 180 injections gives a 95% interval of 98.89–100% for aggregate detection. Twenty benign windows produced 0/543 poll alarms. A secondary live extension completed the exact vectors in 45/45 compound observations, and a 30-minute benign soak produced 0 alarms or epistemic warnings in 4,680 polls. Separate actuation, detection, vector-completion, and evidence clocks retain two transient misses and asynchronous partial diagnoses. This bounded execution demonstrates realizability on one stack with file-backed governance streams; it does not estimate prevalence, production effectiveness, transport reliability, or transfer.

Index Terms—Governance drift, configuration drift, GitOps, policy as code, cloud governance, Kubernetes, continuous compliance, DevOps, distributed systems.

I. INTRODUCTION

Reconciliation-based delivery gave cloud operations a powerful invariant: the running system converges to a declared desired state, and divergence—*configuration drift*—is detected and repaired continuously [1, 2, 3, 4, 5]. The invariant is so useful that it has quietly become a proxy for a much stronger claim. When the reconciler reports SYNCED and the drift detector reports clean, operators, auditors, and dashboards read: *the system is in the state we approved*. This paper is about the

gap between those two statements.

Consider a payments service whose deployment was approved on March 3rd: the manifest matched revision `rev-42`, satisfied policy version π_7 , carried a service-owner approval bound to image digest `sha256:aaa...`, and traced to ticket TKT-99. Months later the cluster still matches its Git manifest byte for byte. Meanwhile: the security team superseded π_7 with π_8 , which the running configuration violates; an emergency exception granted during an incident expired after four hours and has now been silently in force for six weeks; the registry re-tagged `svc:1.4.2` so the pods run a digest no approval ever referenced; and a well-meaning rollback reverted Git to `rev-37`, content whose approval belongs to a different change. Configuration drift: zero. Every governance fact that made the deployment legitimate on March 3rd: gone. We call this condition *governance drift*: the operational state has diverged not from its manifest but from the state, policy context, authorization basis, and intent *under which it was approved*.

The distinction is not terminological. Configuration drift is a *two-place, memoryless* relation between two configurations—desired and observed—and is therefore decidable by comparison, which is exactly what reconcilers and drift tools [6, 7] implement. Governance drift is a *three-place, history-anchored* relation among present state, evolving context, and an immutable admitted basis; we establish the separation in two complementary forms: constructed, tool-realizable counterexamples, and a definitional-dependency result showing that no detector whose inputs are configuration pairs—however sophisticated its diffing—can decide governance consistency, because the deciding facts are not functions of those inputs (Proposition 1). Worse, reconciliation can *manufacture* governance drift while eliminating configuration drift: the rollback scenario above is converged by the reconciler into a state that no current approval covers (Section III).

Our executed study makes the separation concrete. We implement a governance-drift detector evaluated at cumulative evidence tiers—configuration comparison; plus policy re-evaluation; plus authorization records with validity semantics; plus artifact lineage; plus environment inventory against recorded assumptions—against twelve injected drift scenarios (including compound drift and evidence decay) and a no-drift control containing both runtime churn and *benign governance churn* (satisfied policy revisions, approved updates, hygienic exceptions), under a counterfactual scoring rule that credits information rather than alarm volume. The result is a strict detectability staircase (Table II): normalized configuration com-

parison detects exactly one scenario of twelve; each evidence tier detects precisely the scenarios whose deciding facts it carries; the governance tiers achieve zero false alarms *earned against* governance churn. A deliberately naive unnormalized comparator is retained only as a scoring diagnostic, not as a production baseline.

A. Research Questions

- RQ0:** Can divergence between approved intent and operational reality be formally represented and measured as governance drift?
- RQ1:** Which forms of governance drift occur in cloud-native systems?
- RQ2:** Can governance drift be detected using existing operational, governance, and control-plane evidence streams?
- RQ3:** How quickly does governance drift become visible?
- RQ4:** Which forms of drift are detectable through configuration comparison alone, and which require authorization or evidence context?

RQ0 is answered by the formal model and the combined analytic and empirical results. RQ1 is answered by the taxonomy and scenario catalog (Sections III and VI); RQ2 and RQ4 by the dependency result (Proposition 1), the tiered architecture (Section V), the executed detectability study (Section VI), and the bounded live laboratory (Section VII); RQ3 by the study’s event-time measurements and the laboratory’s separated actuation, onset-to-detection, end-to-end, and time-to-evidence observations across three polling cadences.

B. Contributions and Epistemic Status

- C1. Taxonomy** (conceptual): formal definitions separating five substantive components—configuration, policy, authorization, intent, and environment—plus transversal epistemic evidence drift, with governance drift as their umbrella—and with the explicit commitment that governance drift must not, and provably does not, reduce to configuration drift (Section III).
- C2. Formal model** (conceptual/analytic): an immutable sequence of approved-state snapshots, observed state $G_{\text{obs}}(t)$, time-indexed governance contexts $\mathcal{P}(t), \mathcal{A}(t), \mathcal{J}(t)$, and a component-wise governance-divergence vector $\text{GD}_{\text{sub}}(t)$ plus an observability mask $O(t)$, with a reasoned rejection of a primitive scalar distance (Section IV).
- C3. Separation results** (analytic): constructed counterexamples and a definitional-dependency proposition establishing that configuration equality does not imply governance consistency and that no configuration-pair detector decides it—deliberately elementary, and billed as such (Sections III and IV).
- C4. Detection architecture** (design): a vendor-neutral tiered architecture mapping each component to a sufficient ev-

idence tier, together with indistinguishable-world constructions proving tier minimality (Section V and Proposition 3).

- C5. Executed detector study** (empirical, closed-world): twelve scenarios $\times$ six detector tiers $\times$ twenty seeds, with counterfactual scoring, full-vector ground truth, three compound scenarios, exact-set and Hamming scoring, an independent composite baseline without the admitted-basis join, and a control containing runtime *and* benign governance churn; all code and data packaged, matrix generated programmatically (Section VI). Its role is precisely scoped: it validates that the reference semantics realize the model’s dependency structure and quantifies noise behavior; it is not field evidence, and it makes no claim about any real cluster.
- C6. Repeated Kubernetes/GitOps laboratory** (empirical, executed): a pinned Kind/Flux/Kyverno stack, local Git and OCI services, approved-state recorder, tiered evaluator, and executable injections for S1–S9. In one hundred eighty controlled injections (20 per scenario), three randomized-phase polling cadences produced 538/540 detections and 100% conditional exact-set accuracy; 20 balanced benign-change windows produced zero alarms in 543 polls (Section VII). A secondary S10–S12 extension reached the exact vector in 45/45 cadence observations; six 300-s benign windows added 4,680 polls without substantive or epistemic alarms. The execution establishes bounded realizability and within-laboratory behavior, not natural occurrence, prevalence, production effectiveness, stream-loss reliability, or transfer.

Established results are cited; analytic claims are proved in the model; the closed-world and live-stack empirical claims are separated explicitly; and the repeated live-stack protocol is reported with its negative controls and misses rather than presented as production evidence.

C. Scope and Non-Goals

We address drift *after* an approved deployment; admission-time enforcement [8, 9, 10, 11] is complementary and assumed imperfect (break-glass paths exist by design). We propose no product; the detector logic we release is a reference semantics, not a system. And we take no position on *which* governance controls organizations should impose—the model makes drift relative to whatever was approved under whatever policy; the choice of policy is the organization’s.

II. BACKGROUND: THE RECONCILIATION BLIND SPOT

A. Desired-State Delivery and Its Invariant

GitOps-style delivery declares desired state in a versioned repository and runs controllers that continuously converge the runtime toward it [1, 2, 3, 4]; operational practice treats the resulting convergence signal as a primary health indicator [12]. The reconciler’s contract is a fixed-point property over *configurations*: $G_{\text{obs}} \rightarrow \text{desired}$. Drift detection in current

practice—reconciler status, cloud configuration recorders, IaC drift tools [6, 7], posture management [13]—implements variations of the same comparison, and configuration-management guidance institutionalizes it as the control objective [14, 15].

Three properties of this regime matter here. First, the comparison’s *reference* is the current desired state, not the approved state: if the desired state itself changes illegitimately, the reconciler converges to the illegitimate state and reports health. Second, the comparison’s *vocabulary* is configuration: facts that are not configuration—policy versions, approval windows, artifact lineage—are outside its type. Third, the comparison is *memoryless*: it relates two present states and consults no past event, whereas approval is precisely a past event.

B. The Governance Context Moves

Meanwhile, the context that made a deployment legitimate evolves on its own clocks. Policy-as-code is versioned and revised continuously [9, 10]; organizations supersede controls without redeploying workloads, under regulatory regimes that presume change remains authorized over time [16, 15]. Authorizations are temporal by nature—emergency exceptions with windows, approvals with scopes, roles that are granted and revoked [17, 18]; their validity changes with no configuration event at all. Artifact references are mutable (tags move; registries re-tag), so the binding between an approval’s subject and the running binary can rot while every manifest byte stays fixed [19, 20]. And parts of the environment that govern a workload’s risk (IAM scope, network exposure, cloud-resource settings) are frequently *unmanaged*: no desired state exists for them, so no comparison can flag their change.

C. Why This Produces a Blind Spot, Not a Bug

None of the mechanisms above is broken. Each does exactly what it is specified to do; the blind spot is an emergent property of their composition: the only continuously-checked invariant in the system is configuration equality, while the property stakeholders actually care about—*this running state is one that current policy, valid authorization, and recorded intent still cover*—is checked jointly exactly once, at admission, and thereafter only in fragments: policy engines background-scan existing resources against *current* policy [10, 9], configuration recorders track unmanaged resources against their own baselines [6], and continuous-validation features re-check artifact policy [11]—each against its own reference, none against the admitted basis, and no component checking validity clocks on the authorization itself. The general property—joint consistency of running state with current context *and* the admitted basis, with class-resolved verdicts—has, to our knowledge, neither a name, a model, nor a detector semantics, although each fragment above supplies detection machinery one tier of it can reuse (Section V). Naming and modeling it is this paper’s job; Section III begins with the vocabulary.

A terminological note to prevent collisions: “drift” is also used for statistical distribution change in learning systems (concept drift [21]); our subject is unrelated. Industry writing on “cloud governance and drift” uses governance as the *program*

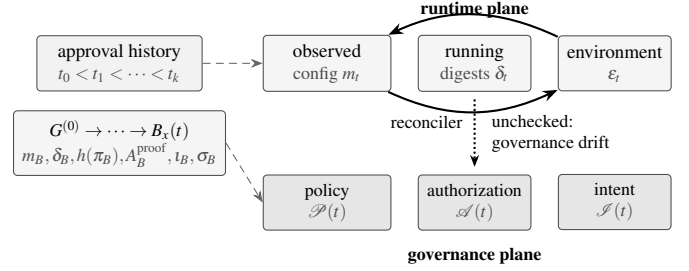


Fig. 1. Approved intent versus runtime reality. The approval event at t_0 freezes a snapshot: configuration, policy version, authorization basis, intent linkage, and environment assumptions. Afterward, two planes evolve independently: the runtime plane (observed configuration, running artifacts, environment) and the governance plane (policy versions, authorization validity, intent records). Reconcilers continuously compare within the runtime plane (solid loop); nothing in standard practice compares the runtime plane against the *evolved* governance plane (dashed gap)—the space in which governance drift lives.

that manages configuration drift [22]; our subject is drift of *governance itself*—a relation the configuration vocabulary cannot express, as Section III now makes precise.

III. A TAXONOMY OF DRIFT (RQ1)

We fix vocabulary relative to an *approval event*: the moment a change was legitimized for an environment. Figure 1 shows the two planes that subsequently evolve. All definitions are made precise in the model of Section IV; here we give their content and their separations.

Definition 1 (Approved State): *The approved state of a deployment is an append-only sequence of immutable snapshots, one per approval event. Each snapshot records: the approved configuration (manifest content and the artifact digests it resolves to), the policy version under which it was evaluated, immutable authorization proof (approvals, exceptions, their subjects, scopes, execution times, and windows), the intent linkage (the change record and revision the approval covers), and the declared environment assumptions within which the risk assessment was made. Evaluation selects the latest applicable covering snapshot; prior snapshots remain evidence rather than being overwritten.*

Definition 2 (Configuration Drift): *Divergence between the current desired configuration and the observed configuration: $G_{\text{obs}}(t) \neq \text{desired}(t)$. This is the classical notion [6, 7, 14], decidable by comparison of present states. It is an operational component of the umbrella vector; by itself it does not assert that either state is unauthorized.*

Definition 3 (Policy Drift): *Divergence between the policy version π_B pinned in the selected admitted basis $B_x(t)$ and the policy context now governing the environment, as it bears on this deployment: $\mathcal{P}(t)$ has changed such that the running configuration would not be admitted under it today, although it was compliant when approved.*

Definition 4 (Authorization Drift): *Invalidation or mismatch of the authorization basis without any configuration event: a*

continuing approval or temporary exception lapsed while its effect persists; an applicable approval was revoked under its declared prospective or retroactive semantics; or the running artifact's digest is not the subject any valid approval refers to. A one-shot approval need only have been valid at execution, provided its integrity-valid proof survives.

Definition 5 (Intent Drift): Divergence between the content now driving the runtime and the intent record the authorization covers: the desired state was changed (e.g., rolled back) to content whose approval basis is absent or belongs to a different change, and reconciliation faithfully executes it.

Definition 6 (Evidence Drift): Decay of the records connecting the running state to its approval basis: the chain (revision $\rightarrow$ artifact $\rightarrow$ deployment $\rightarrow$ approval) can no longer be established from surviving evidence, so governance consistency becomes undecidable rather than false. Evidence drift converts governance questions from answerable to unanswerable. It is therefore an epistemic, transversal class: unlike the five substantive components, it does not assert that a governed relation is inconsistent; it records that one or more such relations can no longer be decided. We keep it inside the governance-drift umbrella because the operational response—repair evidence and withhold a polar verdict—is class-specific.

Definition 7 (Environment Drift): Violation, outside the managed configuration's scope, of an environment predicate recorded by the approval's risk assessment: IAM scope broadened, network exposure altered, an adjacent cloud resource modified out of band—no desired state exists for the changed property, so no configuration comparison can even represent the divergence.

Definition 8 (Governance Drift): The condition in which the operational state of an environment diverges from the state, policy context, authorization basis, intent, or environment assumptions under which it was approved, together with the epistemic condition in which one or more such relations can no longer be decided. Formally, the umbrella contains five substantive divergence components (Definitions 2 to 5 and 7) and one transversal evidence condition (Definition 6). A deployment is governance-consistent at t when every substantive divergence of Section IV is zero and every required component is decidable. The substantive components are not mutually exclusive: one incident may drift several at once (an unapproved rollback drifts intent and leaves the runtime on digests no valid approval covers), which is one reason the model of Section IV keeps them as a vector.

A. Risk Channels and Severity

The five substantive divergence components and the epistemic evidence class are *not* equivalent in ontology or consequence. A raw count therefore cannot stand in for severity. Each class opens a different primary risk channel and implies a different default disposition, as summarized in Table I. Configuration drift first threatens operational integrity; policy drift threatens compliance with the currently governing control; authorization

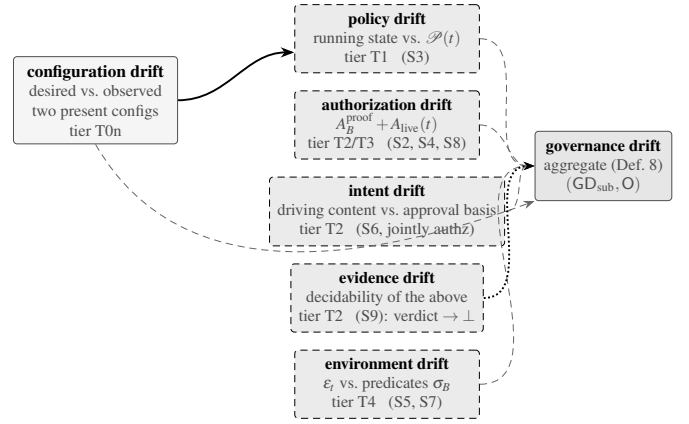


Fig. 2. The drift taxonomy. Configuration drift (left) is a relation between two present configurations. Five substantive components relate the present state to the approved snapshot or evolved context; evidence drift is the transversal loss of decidability shown by the dotted path. Their umbrella is governance drift (Definition 8). Annotated under each class: the evidence tier of Section V that first decides it, as measured in Section VI.

drift threatens security legitimacy; intent drift threatens the legitimacy of the change itself; evidence drift threatens audit and forensic decidability; and environment drift changes the exposure or assumption boundary around the approved system.

This mapping is causal, not ordinal: drift class $\rightarrow$ risk channel $\rightarrow$ response family. Within a class, severity must still be conditioned on asset criticality, affected subjects, exposure time, reversibility, and the confidence permitted by surviving evidence. Across classes, a low-impact authorization mismatch can be less severe than a configuration fault that stops a safety-critical service. The taxonomy thus supports class-specific prioritization without pretending that one universal weight orders every incident.

B. The Central Separation

The taxonomy's load-bearing claim is that Definition 8 does not reduce to Definition 2. We establish it constructively and then in general form.

1) Constructed counterexamples

Figure 3 presents the canonical scenario in full: a deployment whose observed configuration equals its desired configuration at every instant shown, and which passes from governance-consistent to governance-inconsistent three separate times—at the policy supersession (policy drift), at the exception expiry (authorization drift), and at the registry re-tag (authorization drift at digest granularity). In the fourth panel the desired state itself is rolled back and the reconciler *eliminates* the transient configuration drift by converging to content with no valid approval basis: reconciliation manufactures intent drift while restoring configuration consistency. Each panel is realizable with standard tooling, and each is one of the executed scenarios of Section VI.

TABLE I

PROPOSED CLASS-RESOLVED RISK CHANNELS AND ILLUSTRATIVE DEFAULT RESPONSE FAMILIES. THESE ROWS ARE NOT EMPIRICALLY VALIDATED OR A SCALAR RANKING: ACTUAL SEVERITY DEPENDS ON BLAST RADIUS, EXPOSURE DURATION, REVERSIBILITY, CONSEQUENCE, AND EVIDENCE COVERAGE.

Drift class	Primary risk channel	Governance question	Default response family
Configuration	Operational availability and integrity	Does observed state still match the managed desired state?	Reconcile, contain, or accept an explicit runtime exception
Policy	Compliance and control validity	Would the running state satisfy the policy governing it now?	Re-evaluate, migrate under an SLA, or document a current exception
Authorization	Security and privilege legitimacy	Is the running subject still covered by valid authority?	Revoke, block, contain, or page the accountable owner
Intent	Governance and change legitimacy	Does the deployed revision implement the change that was approved?	Halt propagation, restore the authorized revision, or obtain new approval
Evidence	Audit and forensic decidability	Can the approval basis and lineage still be established?	Preserve records, escalate, and report <i>undecidable</i> rather than compliant
Environment	Assumption and exposure boundary	Do unmanaged surroundings still satisfy the recorded risk assumptions?	Contain exposure and reassess the approval in the changed environment

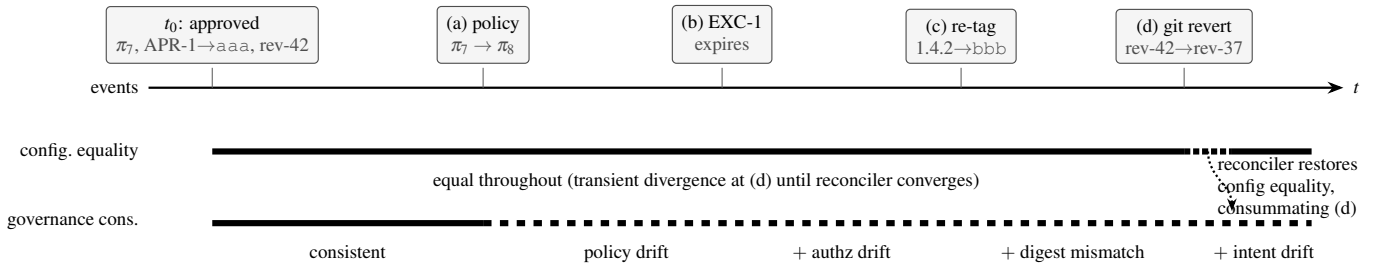


Fig. 3. Configuration-consistent but governance-inconsistent, in four panels. Timeline of one deployment: observed configuration (bottom band) remains equal to desired configuration throughout—configuration drift is zero at every instant after reconciliation—while governance consistency (top band) is broken successively by (a) policy supersession $\pi_7 \rightarrow \pi_8$, (b) expiry of emergency exception EXC-1 whose effect persists, (c) registry re-tag moving $svc: 1.4.2$ to a digest no approval references, and (d) a Git rollback that the reconciler faithfully converges to, eliminating transient configuration drift while stranding the runtime on content whose approval belongs to a different change. Panels correspond to Scenarios S3, S2, S4, and S6 of Section VI.

2) The dependency form

Proposition 1 (Configuration equality does not decide governance consistency): *Let D be any detector whose inputs are configuration pairs ($desired(t), G_{obs}(t)$)—including their full histories. There exist worlds w_1, w_2 with identical configuration-pair histories in which the same deployment is governance-consistent in w_1 and governance-inconsistent in w_2 . Hence no such D decides governance consistency, for any diffing sophistication.*

Proof. Take w_1 and w_2 to agree on all configurations at all times and differ only in the governance plane: in w_2 the policy version governing the environment was superseded after t_0 by one the running configuration violates (or: an exception's window elapsed; or: a valid approval in w_1 is absent in w_2 for the same content). These differences live in $\mathcal{P}(t), \mathcal{A}(t), \mathcal{I}(t)$, which are not functions of the configuration pair; both worlds present D with identical inputs, and Definition 8 assigns them different values. D outputs the same answer in both, so it errs in one. $\square$

The proposition is elementary—a definitional-dependency observation, and we bill it as such rather than as a deep impossibility theorem. Its force is architectural: it says deployed drift detectors are not approximations of governance-drift detectors

but detectors of a different relation, and no incremental improvement of configuration diffing closes the gap as a decision procedure. Two bounds on what it claims: it rules out *deciding*, not statistical *hinting*—in real estates, governance-plane events often correlate with configuration-plane events (policy revisions ship alongside config pushes; rollbacks are visible in Git history), so configuration histories may carry probabilistic signal about governance state; how much is an empirical question for the laboratory, which our closed world deliberately does not answer. And it says nothing against comparison as a *mechanism*: the governance tiers of Section V are themselves comparisons—against recorded bases, validity clocks, and coverage sets—which is precisely the point: the input vocabulary, not the mechanism, is what must widen.

3) What configuration drift remains

The separation does not demote configuration drift: unauthorized manual change (Section VI, S1) is both configuration and governance drift, and configuration comparison is its right detector. The taxonomy's point is containment, not replacement: configuration drift is one of five substantive components under the six-class umbrella, and the only one decidable in the configuration vocabulary—which the study confirms empirically as a detectability staircase whose first step is exactly that one class.

IV. A FORMAL MODEL OF GOVERNANCE DRIFT (RQ2, RQ4)

A. States and Contexts

Fix a deployment x into environment e with approval events at $t_0 < \dots < t_k \leq t$.

Approved-snapshot history. Per Definition 1, every approval event appends an immutable snapshot

$$G_{\text{app}}^x(t) = \{G_{\text{app}}^{(0)}(t_0), \dots, G_{\text{app}}^{(k)}(t_k)\},$$

$$G_{\text{app}}^{(i)} = (m_i, \delta_i, h(\pi_i), A_i^{\text{proof}}, l_i, \sigma_i).$$

Each record contains the approved manifest content m_i and artifact digests δ_i it resolved to at approval; the pinned policy version; the authorization basis A_i^{proof} (the immutable proof that approval instruments covered the named subjects and scope at their execution times); the intent linkage l_i (change record, revision); and the environment assumptions σ_i (the declared properties of e —identity scopes, exposure—under which risk was assessed). Let $b(t)$ select the greatest i such that $t_i \leq t$ and the snapshot scope covers (x, e) . The admitted basis is

$$B_x(t) = G_{\text{app}}^{(b(t))} = (m_B, \delta_B, h(\pi_B), A_B^{\text{proof}}, l_B, \sigma_B).$$

Selection does not require current validity: an expired or revoked basis must remain selected so its divergence can be reported. Superseded snapshots remain available for audit and lineage; benign reapproval appends a new basis rather than mutating history. Append-only means records cannot be rewritten, not that their storage cannot be lost; loss of A_B^{proof} is an evidence failure, as S9 exercises.

Observed state. $G_{\text{obs}}(t) = (m_t, \delta_t, \epsilon_t)$: the observed configuration, the digests actually executing, and the observed environment properties needed to evaluate the predicates in σ_B .

Contexts. Three time-indexed context streams: $\mathcal{P}(t)$, the policy version(s) governing e at t with their evaluation semantics; $\mathcal{A}_{\text{live}}(t)$, the authenticated current administrative state of instruments (revocations, continuing validity, expiry); and $\mathcal{I}(t)$, the intent records and the desired-state revision currently driving delivery. Evidence availability, where needed, is a fourth stream $\mathcal{E}(t)$ (the records from which the above can be established).

Authorization semantics. Every instrument a declares $\text{mode}(a) \in \{\text{one-shot}, \text{continuing}, \text{temporary-exception}\}$, subject, scope, validity interval, and revocation policy. One-shot authority is required at execution and its proof is retained afterward; continuing authority must remain valid while its effect persists; a temporary exception is continuing only until its expiry. Revocation is prospective unless the record explicitly declares retroactive invalidation. Let $\text{NeedsLive}(a)$ hold for continuing and temporary instruments, and for a one-shot instrument only when its declared revocation policy permits retroactive invalidation. For an effect executed at τ , let $\text{AppObs}(a, t)$ mean that its proof is available and, whenever $\text{NeedsLive}(a)$ holds, authenticated live status is also available. Write V_1 for

validity at τ without retroactive revocation, V_c for current validity without revocation, and V_e for V_c while the declared exception has not expired. Define the three-valued applicability function

$$\text{Applicable}(a, \tau, t) = \begin{cases} \perp, & \neg \text{AppObs}(a, t); \\ V_1(a, \tau, t), & \text{one-shot}; \\ V_c(a, t), & \text{continuing}; \\ V_e(a, t), & \text{temporary exception}. \end{cases}$$

A retained historical proof is therefore not confused with currently effective authority. Missing proof or live status yields undecidable where the selected mode requires it; an authenticated revocation yields a polar inconsistent verdict.

Legitimate successors. Let $\text{Succ}(l_B, t)$ be the set of revisions l_n for which there exists a finite chain $l_B, l_1, \dots, l_n$ with transition execution times $\tau_1, \dots, \tau_n$, such that every edge (l_{j-1}, l_j) and its artifact digest are covered by an instrument a_j for which $\text{Applicable}(a_j, \tau_j, t)$ holds. A revision is a legitimate successor iff it belongs to this set. This definition excludes mere Git ancestry: lineage without a valid approval edge does not extend the admitted basis.

B. The Governance-Divergence Vector

We define one divergence component per substantive relation; each is a function of exactly the inputs its component needs, which is what makes the detectability results of Section VI possible to state.

- $d_{\text{conf}}(t)$: a normalized difference over the managed configuration graph between $\text{desired}(t)$ and m_t (field-level graph diff with runtime-owned fields factored out; Section VI shows the normalization is not cosmetic but the difference between signal and noise). This is an operational consistency component included to give the evaluator a complete control-state report; nonzero d_{conf} alone does not imply that either the desired or observed state lacks authorization coverage.
- $d_{\text{pol}}(t)$: the set of controls in $\mathcal{P}(t)$ that the running state violates although $B_x(t)$ records compliance under π_B —empty iff admission would re-succeed under today’s policy. Reported as a violated-control set, not a number: policy versions form a revision structure, not a metric space, and “how far” is policy-weighted judgment, not geometry.
- $d_{\text{auth}}(t)$: the set of relied-upon instruments for which $\text{Applicable}(a, \tau, t)$ is false, plus running digests in δ_t not covered by an applicable instrument grounded in A_B^{proof} and $\mathcal{A}_{\text{live}}(t)$. If applicability is $\perp$, the authorization mask entry is zero and the substantive component is undecidable rather than included in this set.
- $d_{\text{int}}(t)$: the mismatch between the revision driving delivery and the revision the valid authorization covers: the current revision is outside $\{l_B\} \cup \text{Succ}(l_B, t)$.
- $d_{\text{env}}(t) = \{q \in \sigma_B : q(\epsilon_t) = \text{false}\}$. The laboratory’s declared IAM-scope and exposure fields instantiate equality

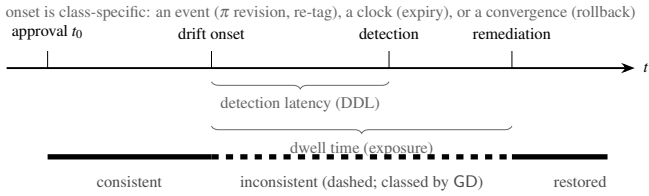


Fig. 4. The drift lifecycle and its temporal quantities (Definition 9). Onset is class-specific—an organizational event (policy revision), a clock crossing (exception expiry), an exogenous event (registry re-tag), or a reconciler convergence (rollback). Detection latency is the tier-dependent quantity measured in Section VI; dwell time is the exposure window that remediation closes, with the remediation family determined by the drifted component (Section IV-C).

predicates, but the definition permits ranges, membership, and monotone safety predicates.

Evidence availability is ontologically different from those distances. Define the observability mask $O(t) = (o_{\text{conf}}, o_{\text{pol}}, o_{\text{auth}}, o_{\text{int}}, o_{\text{env}})$, where $o_j = 1$ iff every required item in the component’s evidence contract is present, complete, fresh within its declared bound, integrity-valid, source-authenticated, linked to subject x , and delivered by a healthy stream. Confidence is therefore not an unscoped probability: it is a named contract and threshold published with the evaluator. A missing or contract-failing record yields *undecidable*; an authenticated record stating “revoked,” “negative,” or “never issued” is available evidence and may yield *inconsistent*. Evidence drift is a transition of any required o_j from one to zero. Each substantive component therefore has three-valued reporting semantics—consistent, inconsistent, or undecidable—but only the first two are values of the substantive relation.

Definition 9 (Governance Divergence Vector; Consistency): $\text{GD}_{\text{sub}}(t) = (d_{\text{conf}}, d_{\text{pol}}, d_{\text{auth}}, d_{\text{int}}, d_{\text{env}})(t)$ and $\text{GD}(t) = (\text{GD}_{\text{sub}}(t), O(t))$. *Operational consistency* $\text{OpCons}(x, t)$ holds iff $d_{\text{conf}} = 0$ and its evidence is decidable. *Governance coverage* $\text{GovCov}(x, t)$ holds iff $d_{\text{pol}} = d_{\text{auth}} = d_{\text{int}} = d_{\text{env}} = 0$ and their mask entries are one. *Total Consistent* $(x, t) = \text{OpCons}(x, t) \wedge \text{GovCov}(x, t)$. *Onset of governance drift* is $\inf\{t > t_0 : \neg \text{Consistent}(x, t)\}$; *dwell time* is the duration of inconsistency; *detection latency* is detection time minus onset.

C. Why the Distance Is a Vector, Not a Scalar

A scalar GD was the natural first proposal, and we reject it as primitive for three reasons. *Incommensurability*: the components take values in different structures (graph diffs, control sets, validity states, three-valued verdicts); any scalarization imports policy weights, and those weights are governance choices, not properties of the system. *Remediation asymmetry*: the components have different remediations—reconcile (d_{conf}), re-evaluate and re-approve or migrate (d_{pol}), re-authorize or terminate the exception (d_{auth}), restore or re-approve intent (d_{int}), restore assumptions or re-assess (d_{env}), repair evidence where O is zero—so collapsing them destroys exactly the information an operator needs. *Temporal structure*: components drift

on different clocks (policy revisions are discrete organizational events; exception expiry is a scheduled instant; registry re-tags are exogenous), so a scalar time series conflates processes that must be attributed separately. A weighted composite for reporting is legitimate *on top of* the vector (Section VIII), with published weights, exactly as a risk score summarizes but does not replace findings.

D. Detectability, Formally

Each component divergence is a function of specific streams: d_{conf} of (desired, G_{obs}); d_{pol} additionally of $\mathcal{P}(t)$ and π_B ; d_{auth} of A_B^{proof} , $\mathcal{A}_{\text{live}}(t)$, execution times, and δ_i ; d_{int} of $\mathcal{I}(t)$ and ι_B ; d_{env} of ε_t and σ_B ; and every entry of O of the evidence needed for its corresponding computation. Proposition 1 is the negative direction of this dependency structure; the positive direction is equally direct:

Proposition 2 (Tier sufficiency): *A detector with access to the streams a component depends on decides that component’s consistency (in the closed-world model, exactly; in the field, up to the fidelity of the streams). Consequently a sufficient detector for full governance consistency joins the configuration comparison with the policy, authorization, intent/lineage, and environment streams—the tiered architecture of Section V, whose scenario-to-tier realization Section VI checks cell by cell.*

Proof. Each component divergence in Section IV-B is defined as a computable function of the named streams; providing the streams provides the function’s inputs. $\square$

The proposition is an input-enumeration result; whether a concrete implementation realizes it is a separate, checkable question—Section VI performs exactly that check on our released reference semantics, and *only* that check: agreement there validates the implementation, not the model.

Proposition 3 (Tier minimality): *For every stream added at T1–T4, there exist two worlds that are indistinguishable to every lower tier but have different governance verdicts for a component first decidable from that stream. Therefore no lower tier decides that component in general.*

Proof. Construct the pairs while holding all lower-tier observations fixed. For T1, keep desired and observed configuration identical and let the current policy admit the deployment in one world but reject it in the other. For T2, keep the T1 projection fixed and let the same effective exception be valid in one world but expired in the other; alternatively, retain versus delete its approval record, changing the authorization/intent mask from decidable to undecidable. For T3, keep the manifest tag and every T2 stream fixed but let the running tag resolve to an approved digest in one world and an uncovered digest in the other. For T4, keep all T3 streams fixed and change only an unmanaged environment property recorded in σ_B . Each pair has identical lower-tier projections and different component verdicts, so any lower-tier detector returns the same value in both and errs in one. $\square$

Unlike Proposition 2, this is a necessity result: the staircase is not merely a convenient implementation decomposition. Its scope remains relative to the declared component semantics; adding a genuinely equivalent stream at a lower tier widens that tier’s projection rather than refuting the result.

Two modeling notes bound the claims. First, *granularity*: whether the registry re-tag scenario is “configuration” drift depends on reference semantics—under tag semantics the configuration is unchanged; under digest semantics it changed. The model takes no side: it requires only that $B_x(t)$ record resolved digests, which relocates the question from metaphysics to evidence (what did the approval bind?), where Section VI shows it is decidable at the lineage tier. Second, *legitimate context change*: policy supersession does not make drift *culpable*—the deployment was legitimate under π_7 and the organization changed the rules. The model deliberately measures divergence, not blame; whether policy drift triggers migration, exemption, or re-approval is a governance decision the vector informs (Section X).

V. A TIERED DETECTION ARCHITECTURE

Propositions 1 and 2 together prescribe the architecture: governance-drift detection is configuration comparison *plus* the minimal additional evidence streams, joined against the selected admitted basis. Figure 5 organizes it as cumulative tiers, each named for what it adds and each mapped to the drift classes it first decides.

A. Components

Approved-state recorder. At every approval, append an immutable snapshot to G_{app}^x —manifest content, *resolved* digests, policy-version pin, immutable authorization proof with execution times, intent linkage, environment assumptions—and persist it durably and independently of the delivery platforms. This is the architecture’s one hard prerequisite: without a recorded approval basis there is nothing to drift *from*; where such records are absent or decayed, drift verdicts degrade to undecidable (the corresponding entries of O are zero), which the evaluator must report as such rather than as consistency.

Evidence tiers. $T0/T0n$: the desired/observed comparison every reconciler already performs [3, 4, 6]; $T0n$ normalizes runtime-owned fields (replica counts under autoscaling, rollout hashes)—the study quantifies why the naive form is unusable as a governance signal. $T1$: the policy-version stream from the policy engine’s repository [9, 10]; the evaluator re-evaluates the running state against $\mathcal{P}(t)$ and compares with the pinned basis. $T2$: authorization records with mode and validity semantics—expiry, scope, prospective or retroactive revocation—so that exceptions and approvals are objects with clocks, not booleans; ITIL-style change records [18] and platform approvals both map here. $T3$: artifact lineage—digest-level linkage from running workloads to approval subjects, the slice pioneered by supply-chain attestation [19, 20, 23, 11]. $T4$: environment inventory—observation of the unmanaged properties assumed at approval (IAM scope, exposure) and comparison against the *recorded* predicates σ_B in $B_x(t)$, from cloud inventories, audit

streams, and telemetry correlation [24, 13, 25]. Mechanically this is baseline comparison—CSPM’s move—and we say so plainly; the difference is the reference (the approval event’s recorded assumptions rather than a benchmark) and the joined, class-resolved verdict.

Evaluator. A continuous process (or scheduled sweep) computing $GD(t)$ per deployment: cheap incremental checks on context-change events (policy revision published, exception clock elapsed, registry event) rather than full re-scans—governance drift, unlike configuration drift, is mostly *event-driven* on the governance plane, which is why several scenarios in Section VI are detectable with zero latency at the tier that carries the relevant event.

Analytic scaling envelope. For n deployments with at most r policy predicates, a relied-upon instruments, d running digests, and q environment predicates each, a full sweep is $O(n(r + a + d + q))$ time and stores $O(n(s + a + d + q))$ evidence for s retained snapshots. The intended execution is incremental: policy-to-deployment, subject-to-approval, digest-to-workload, and predicate-to-resource indices restrict an event to its k affected deployments, yielding $O(k(r + a + d + q))$ work. Policy results and immutable proofs are cacheable; evaluators can shard by cluster, namespace, or tenant. Kubernetes watches and batched cloud-inventory deltas avoid one API request per deployment. Cloud quotas and cache invalidation remain engineering constraints, not measured results; this laboratory does not claim a throughput bound.

Minimal adoption path. The smallest useful deployment is $T0n+T1+T2$: normalize the reconciler comparison, re-evaluate running state under current policy, and join durable approvals and exceptions with explicit validity semantics. This separates operational, compliance, and authorization risk immediately; $T3$ adds digest lineage and $T4$ adds declared environment predicates as evidence maturity grows. GCC is reported from the first rollout so unevaluable coverage is visible rather than treated as conformance.

For legacy workloads, a discovery pass may create a signed *retrospective baseline* from current state, current policy results, and an accountable owner’s attestation. It is timestamped and labeled as reconstructed evidence—never represented as a historical approval—and GCC remains partial for facts that cannot be recovered. New changes then enter the ordinary append-only path, allowing coverage to increase without rewriting the estate’s past.

B. Relation to Existing Mechanisms

The architecture is deliberately assembled from streams that already exist in mature platforms; its novelty is the *join semantics* against a recorded approval basis and the class-resolved verdict. Admission control [8, 9, 10] evaluates the same predicates once, at entry; continuous-validation features [11] re-evaluate the artifact-policy slice; posture management [13] scans environment configuration against benchmark policy without any approved-state reference. Indeed, per-tier continuous detection largely *exists* in cited tooling, and the architecture credits it explicitly: Kyverno’s background scan-

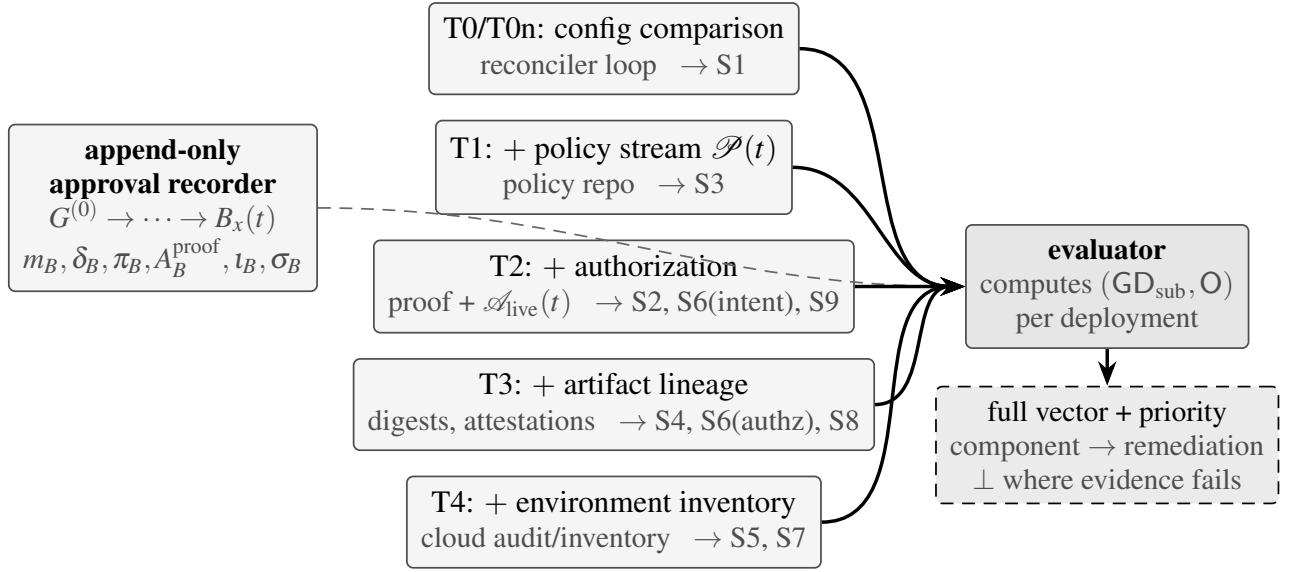


Fig. 5. Tiered governance-drift detection. The append-only recorder (left) preserves successive approvals and selects $B_x(t)$. Evidence tiers (center) cumulatively widen the detector’s vocabulary: T0/T0n configuration comparison (the reconciler’s existing loop, naive or churn-normalized), T1 policy-version stream, T2 authorization records with validity semantics, T3 artifact lineage, T4 environment inventory. The evaluator (right) recomputes the component divergences of Section IV-B continuously and emits classed drift vectors plus an epistemic mask, with a priority projection and the remediation family each component implies. Stream adapters also report the presence, completeness, freshness, integrity, authenticity, subject linkage, and health required by each component’s evidence contract. Annotations show which components of Scenarios S1–S9 each tier first detects, as measured in Section VI.

ning re-evaluates existing resources against current policy (our T1) [10]; configuration recorders track unmanaged resources (much of T4) [6, 13]; continuous validation re-checks artifact policy (part of T3) [11]. We found no documented mechanism that provides the complete join: all tiers evaluated against the *admitted basis* $B_x(t)$, with mode-aware validity on the authorization itself, and verdicts that distinguish “violates policy” from “was never approved” from “was admitted legitimately and the world moved”—the distinctions that determine remediation (Section IV-C). The contribution is the relation and its join semantics, not the comparison machinery, and Section IX restates novelty in exactly those terms.

VI. EXECUTED STUDY: DETECTABILITY ACROSS EVIDENCE TIERS (RQ2–RQ4)

We now measure the taxonomy’s detectability structure. The study is executed and deterministically reproducible from the packaged source (≈ 380 lines of dependency-free Python; the results table, including its header, is generated programmatically by the same run). Its epistemic scope is stated plainly: it is a *closed-world evaluation of detector semantics*—the world, its scenarios, and the detectors are all models—so it establishes what each evidence tier can and cannot decide *in principle*, with event-time latencies. It does not measure any real cluster; wall-clock behavior on real infrastructure is the laboratory’s job (Section VII).

A. Modeled World and Scenarios

The world models one deployment with the selected admitted basis of Section IV-A—including recorded environment assumptions σ_B —and streams for desired state (Git), observed

cluster state, registry contents, policy versions *with evaluable requirement sets* (so that T1 implements Definition 3’s admission re-evaluation, not a version-string comparison), authorization records as a set with subject digests, covered revisions, windows, and revocation (so that legitimate successors are modeled), and two unmanaged environment properties. Each run is 400 events; drift is injected at event 150. Twelve scenarios follow the taxonomy and the constructed examples of Figure 3:

- S1** manual in-cluster change (observed field diverges from desired);
- S2** expired emergency authorization (exception granted with a 40-event window; nothing removes it at expiry);
- S3** policy version change post-deploy ($\pi_7 \rightarrow \pi_8$, which the running configuration violates);
- S4** artifact substitution (same tag re-pointed to a new digest, rolled onto nodes; manifests unchanged);
- S5** IAM permission expansion out of band (unmanaged);
- S6** Git rollback without new approval (reconciler converges to the reverted content; approval binds the newer revision);
- S7** out-of-band cloud console modification (unmanaged);
- S8** approval mismatch (deployed digest is not any approval’s subject);
- S9** live-status deletion for a continuing approval: immutable A_B^{proof} survives, but the authenticated source required to establish continuing validity is lost; authorization and intent become undecidable—the missing status is neither

a revocation nor proof that approval never existed (evidence drift—Definition 6—exercised directly);

S10 policy supersession plus an expired exception ({policy, authorization});

S11 artifact substitution plus environment change ({authorization, environment});

S12 rollback plus deletion of its approval basis: the underlying intent/authorization relations become undecidable, so the honest observable output is evidence drift rather than a guessed polar set;

plus **S0**, a no-drift control. Churn is of two kinds, both present in every stream including the control: *runtime churn* (30% of events: autoscaler replica changes, rolling-restart hash changes) and *benign governance churn* (2% of events: policy revisions the running state satisfies, approved updates that legitimately advance revision, digest, and approval together, and hygienic exceptions removed at expiry). The governance churn exists so that the governance tiers’ false-alarm rates are earned against realistic governance noise rather than measured against a silent plane.

B. Detectors and Counterfactual Scoring

Six detector tiers implement Section V: T0 (naive configuration comparison, including runtime-owned fields), T0n (normalized), and cumulative T1–T4, each a pure function of its tier’s visible streams, so tier capabilities are enforced by construction. Ground truth is a *class set* per scenario (components are a vector and may drift together, Definition 8). The detector exports the complete class set decidable at its tier; a priority-ordered first verdict is derived only as an operational interface. We score exact-set agreement, subset-only partial diagnosis, and Hamming loss rather than accepting any one member of the ground truth. Missing approval records set the relevant entries of O to zero and produce the explicit undecidable/evidence verdict, never a guessed polar one. S6, S10, and S11 test substantive composition; S12 tests composition under evidence loss.

Scoring required care, and the naive arm is why. A detector that alarms constantly “detects” every scenario by coincidence; volume is not information. We therefore score *counterfactually*: for each (scenario, tier, seed), the detector runs on the drift stream and on a paired, same-seed, no-drift control stream with identical churn; the drift is *detected* iff the two alarm sequences differ, detection time is the first differing event, and classification is the class set emitted there. Under this rule a constant alarmer carries zero information and scores zero. False alarms are counted on the control stream. Twenty seeds per cell; onset for S2 is the expiry instant (the drift is the exception *outliving* its window, not its grant).

C. Results

Table II is the study’s central artifact. Four findings, stated in the register the design supports.

Finding F1 (The staircase: implementation validation of tier dependencies): Detection forms a strict staircase realizing Propositions 2 and 3 cell for cell: T0n detects S1 and nothing else—one scenario of twelve; T1 adds policy drift (S3) and the policy component of S10, and—because it re-evaluates policy rather than comparing versions—raises no alarm on satisfied policy revisions in the churn. T2 adds the validity- and coverage-dependent scenarios S2, S6, and S10, plus the evidence-decay scenarios S9 and S12 (undecidable, not polar). T3 adds the digest-level scenarios S4, S8, and S11 and completes S6’s two-component diagnosis; T4 adds S5/S7 and completes S11. The triangle cells show why first-label accuracy was insufficient: a lower tier can report a correct subset without deciding the full vector. Every reported component is correct in every seed, and exact-set agreement appears when the tier carries all required streams. We state this finding’s epistemic value precisely: the world and detectors are co-designed, so the exactness is expected, and the staircase is implementation validation of the model’s dependency structure plus a noise-behavior measurement—not field evidence for the taxonomy, a role we do not claim for it. Its concrete content for RQ4 stands: in the reference semantics, configuration comparison decides only its own substantive component; the five-class substantive vector plus epistemic mask requires the joined tiers. All six umbrella classes are exercised.

Finding F2 (A deliberately naive comparator validates the scoring rule): The unnormalized diagnostic comparator T0 alarms on 285.4 of 400 control events (71.4% in our churn model)—and the rate is nearly flat across runtime churn rates of 10–50% (287, 285, 300 alarms per run), because alarms track the dwell of divergent runtime-owned state, not the event rate. Under counterfactual scoring T0 still detects S1, but with mean latency 22.6 events versus 0 for T0n: its information is buried in its own noise. For the other eleven scenarios it carries zero information while alarming continuously. T0 is not an industry baseline—production reconcilers normalize—and no comparative effectiveness claim rests on it; its sole role is to demonstrate that the counterfactual scoring rule does not reward alarm volume. Conversely, the governance tiers’ zero false alarms are now earned: the control contains satisfied policy revisions (which a version-comparison T1 would have flagged), legitimate approved updates (which a pinned-revision intent check would have flagged), and hygienic exceptions—and the definition-faithful detectors correctly ignore all of them.

Finding F3 (Governance drift is event-detectable, with one designed exception): At the correct tier, every scenario is detected with latency 0 events (including S2 under expiry-inclusive semantics) and none requires scanning: the deciding facts arrive as events on governance streams (a policy revision, a clock crossing, a registry event, an inventory delta). RQ3’s answer in the closed world is therefore: visibility is bounded by evaluation cadence on the governance plane, not by any intrinsic latency of the phenomenon—which sharpens what the laboratory must measure (wall-clock cadence effects, Section VII).

TABLE II

DETECTABILITY MATRIX, GENERATED PROGRAMMATICALLY FROM THE RUN OUTPUTS (20 SEEDS PER CELL). ✓ = DETECTED WITH EXACT-SET AGREEMENT IN ALL SEEDS; △ = DETECTED BUT ONLY A CORRECT SUBSET IS DECIDABLE AT THAT TIER (SUPERScript: MEAN DETECTION LATENCY IN EVENTS WHEN >0); × = NOT DETECTED (NO INFORMATION UNDER COUNTERFACTUAL SCORING). FP = FALSE ALARMS PER 400-EVENT CONTROL RUN.

Scenario	T0	T0n	T1	T2	T3	T4
	naive cfg	norm. cfg	+policy	+authz	+lineage	+env
S0 control (churn only)	FP: 285.4	FP: 0.0	FP: 0.0	FP: 0.0	FP: 0.0	FP: 0.0
S1	✓ +22.6	✓	✓	✓	✓	✓
S2	×	×	×	✓	✓	✓
S3	×	×	✓	✓	✓	✓
S4	×	×	×	×	✓	✓
S5	×	×	×	×	×	✓
S6	×	×	×	△	✓	✓
S7	×	×	×	×	×	✓
S8	×	×	×	×	✓	✓
S9	×	×	×	✓	✓	✓
S10	×	×	△	✓	✓	✓
S11	×	×	×	×	△	✓
S12	×	×	×	✓	✓	✓

S1 manual change; S2 expired exception; S3 policy supersession; S4 artifact substitution; S5 IAM expansion; S6 unapproved rollback; S7 out-of-band cloud change; S8 approval mismatch; S9 approval-record deletion; S10 policy+exception; S11 artifact+environment; S12 rollback+missing approval. Tiers are cumulative; T0n normalizes runtime-owned fields; latencies use expiry-inclusive semantics.

Finding F4 (Reconciliation health is compatible with total governance failure): In eleven of twelve scenarios, post-convergence normalized configuration comparison reports clean—while the deployment runs under superseded policy, expired authorization, unapproved content, or altered environment. This is Proposition 1 made operational, and it quantifies the blind spot claimed in Section II: a SYNCED reconciler is evidence about configuration, and about configuration only.

D. Independent Composite Baseline: Signals Without the Join

The staircase establishes dependency realization but does not isolate the value of the admitted-basis join. We therefore execute a second baseline that unions five plane-local mechanisms: normalized desired/observed comparison, current-policy evaluation, exception-expiry checking, digest membership in a tool-local attestation trust set, and current environment posture against a benchmark. It receives the same event stream and benign churn as T4 but has no $B_x(t)$, snapshot selection, intent history, or evidence-state semantics. This is a stronger baseline than T0 and represents composition of existing signals without claiming to reproduce a particular commercial product.

The local union exactly identifies 9/12 scenario sets (180/240 scenario-seed units, 75%); joined T4 identifies 12/12 (240/240, 100%). Both remain quiet on the paired controls, so the difference is diagnosis, not alarm volume. The local union misses S6’s intent component, cannot see S9’s evidence loss,

and mislabels S12 as an authorization finding rather than withholding a polar verdict. Four semantic probes sharpen causality: identical current-policy failure with versus without an admitted basis produces the same local “policy” alert, while the join returns policy versus evidence; missing live status yields evidence for a continuing instrument but remains consistent for a prospective one-shot instrument with retained proof. The incremental value is therefore the history- and mode-aware relation that selects both class and remediation family. As with the staircase, this closed-world comparison validates semantics, not field effectiveness.

E. Threats to Validity

Construct: detectors and world share a modeling vocabulary; the tiers’ *exact* 0/1 staircase is a property of the closed world, where streams are noiseless and semantics are implemented as defined. Real streams are late, lossy, and ambiguous; the staircase’s *shape* (which class needs which evidence) is the transferable claim—it follows from input dependencies (Proposition 2)—while the crispness is not. *Internal:* single-deployment world and one injection time per run. Three compound scenarios exercise set output, but do not span the combinatorial space. The first-verdict order is one operational policy among several; exact-set scoring is invariant to it. Scenario S6’s transient configuration signal (before the reconciler converges) is not credited to T0n because the comparison evaluates

TABLE III
COMPOSITE PLANE-LOCAL BASELINE VERSUS THE ADMITTED-BASIS JOIN. EXACT COLUMNS ARE PERCENTAGES OVER 20 SEEDS; EMPTY LOCAL OUTPUT ON S9 MEANS THE DELETED GOVERNANCE STREAM IS OUTSIDE ITS VOCABULARY.

Scenario	Expected	Local union	Joined	Local exact	Joined exact
S1	{ <i>config</i> .}	{ <i>config</i> .}	{ <i>config</i> .}	100	100
S2	{ <i>auth</i> .}	{ <i>auth</i> .}	{ <i>auth</i> .}	100	100
S3	{ <i>policy</i> .}	{ <i>policy</i> .}	{ <i>policy</i> .}	100	100
S4	{ <i>auth</i> .}	{ <i>auth</i> .}	{ <i>auth</i> .}	100	100
S5	{ <i>env</i> .}	{ <i>env</i> .}	{ <i>env</i> .}	100	100
S6	{ <i>intent</i> , <i>auth</i> .}	{ <i>auth</i> .}	{ <i>intent</i> , <i>auth</i> .}	0	100
S7	{ <i>env</i> .}	{ <i>env</i> .}	{ <i>env</i> .}	100	100
S8	{ <i>auth</i> .}	{ <i>auth</i> .}	{ <i>auth</i> .}	100	100
S9	{ <i>evidence</i> .}	{ }	{ <i>evidence</i> .}	0	100
S10	{ <i>policy</i> , <i>auth</i> .}	{ <i>policy</i> , <i>auth</i> .}	{ <i>policy</i> , <i>auth</i> .}	100	100
S11	{ <i>auth</i> , <i>env</i> .}	{ <i>auth</i> , <i>env</i> .}	{ <i>auth</i> , <i>env</i> .}	100	100
S12	{ <i>evidence</i> .}	{ <i>auth</i> .}	{ <i>evidence</i> .}	0	100

post-convergence state in our event ordering; on real clusters a fast comparator might glimpse it—a cadence question for the lab. *External*: the repeated laboratory (Section VII) shows that the nine scenario classes can be induced and usually observed through real Kubernetes/GitOps interfaces on one pinned stack. It does not transfer the closed-world rates quantitatively or estimate natural occurrence, field prevalence, stream-loss reliability, multi-cluster behavior, or performance under load. Its two cadence misses are direct evidence against assuming that polling necessarily observes transient configuration drift.

VII. REPEATED KUBERNETES/GITOPS LABORATORY

The closed-world study fixes semantics; a live stack tests whether the declared classes and evidence dependencies are concretely realizable through control-plane interfaces. We therefore executed a repeated laboratory over the nine atomic scenarios and six benign controls, followed by a smaller compound-vector extension and a 30-minute benign soak. Its purpose is deliberately bounded: realizability and within-laboratory detector behavior on one pinned stack. It is not evidence of natural occurrence, field prevalence, production effectiveness, or transfer across organizations.

A. Laboratory Hypotheses

The experiment tests three predeclared, falsifiable hypotheses. **LH1 (realizability)**: each injected scenario, when observed at or above its predicted minimum evidence tier, yields exactly the class set in the model. **LH2 (cadence sensitivity)**: a transient drift interval can be missed when its dwell time is shorter than the evaluator cadence. **LH3 (specificity within scope)**: admitted-basis-preserving changes in the six benign families produce neither substantive drift nor epistemic warnings. These are laboratory hypotheses, not prevalence or cross-platform claims.

B. Build and Reproducibility Boundary

A single-node Kind v0.32.0 cluster running Kubernetes v1.36.1 [26] hosts Flux v2.9.4 [4] and Kyverno v1.18.2 [10]. A local smart-HTTP Git service drives Flux and a local OCI registry supports tag/digest substitution.

Approval records and the environment inventory are versioned JSON files, not production approval or cloud-inventory services. Bootstrap pins upstream manifests by SHA-256, preloads the required arm64 images, and records platform versions. An approved-state recorder snapshots G_{app} : manifest, resolved pod-image digest, policy pin, approval JSON with validity window, Git revision, and declared environment assumptions. The dependency-free evaluator reads Kubernetes, Git, Kyverno PolicyReports, file-backed live authorization status, pod image IDs, and the file-backed inventory through tier-restricted entry points T0n–T4. The bootstrap, injections, repeated harness, evaluator, raw observations, controls, and generated tables ship under `lab/`. The baseline deployment uses a continuing approval: its immutable proof is captured in $B_x(t)$, while the live JSON supplies current administrative status; S9 removes the latter and therefore makes continuing validity undecidable. In this bounded implementation, stream health and syntactic completeness are checked at read time and resource names provide subject linkage; local files are trusted sources, so cryptographic source authentication and independent freshness attestations remain outside the executed contract. The reference semantics are explicit: T0n compares the declared image reference (including its tag) as configuration, while T3 resolves the running `imageID` digest and tests whether an applicable approval covers that digest. Thus re-tagging can remain configuration-consistent yet fail lineage.

Measurements ran on an arm64 Apple M4 Max host with 48 GiB RAM and macOS 26.5.1, Docker 29.1.3, and Python 3.11.9. The Kind node reported 16 CPUs and 8,024,028 KiB memory; Docker imposed no explicit per-container CPU or memory limit. No benchmark workload ran concurrently, but ordinary host background services were neither disabled nor load-controlled. Durations use Python’s monotonic clock (`mach_absolute_time()`), reported resolution 4.17×10^{-8} s, so wall-clock adjustment cannot invert intervals. These conditions make the reported seconds interpretable as measurements of this host, not portable performance guarantees; the machine-readable snapshot is stored with the observations.

C. Protocol and Four Clocks

For each repetition, a seeded shuffle changes scenario order. Before every injection, the harness restores the approved Git revision, policy π_7 , approval record, artifact tag, deployment, and inventory, and requires T4 to return *consistent*. Each S1–S9 scenario is executed 20 times. Three logical full-vector evaluators observe the same injection at cadences 0.5, 2, and 10 s; their first post-onset phase is sampled independently and deterministically from $[0, c)$ for cadence c , after which they poll every c seconds. Thus a latency below 0.5 s is possible when the first scheduled poll falls shortly after onset; 0.5 s is a cadence, not a measurement-resolution floor. We report two decimal places and retain higher-resolution raw clocks only for recalculation.

The experimental unit is the injected scenario repetition (180 units: 20 per scenario). The 540 cadence-specific observations are correlated repeated measurements over those same injections, not 540 independent experiments. We therefore report Wilson intervals separately by cadence and use a seeded non-parametric bootstrap that resamples the 180 injections as clusters for the aggregate rate.

The harness records four timestamps: command initiation t_{inject} , operational inconsistency t_{onset} , first non-consistent full-vector verdict t_{detect} , and completion of the minimal JSON evidence bundle t_{evidence} . It reports actuation, DDL, end-to-end latency, and TTE exactly as defined in Metric M2. For command-carried mutations, onset is conservatively operationalized as successful command completion. For S2 it is exception expiry; for S4, successful rollout of the substituted artifact; and for S6, successful Flux convergence to the unapproved revision. Consequently, S6’s Git commit and reconciliation belong to actuation and end-to-end time, not DDL. S3 begins at successful policy supersession, so its DDL includes waiting for Kyverno’s background PolicyReport. Evidence assembly in this reference implementation is synchronous with verdict serialization, so TTE and DDL are nearly identical; this does not estimate retrieval delay for a platform-scale evidence service.

Ground truth is a set. S6 is $\{\text{intent}, \text{authorization}\}$; T2 can expose the intent component, while T3 lineage is required to decide its authorization component. Every live observation is therefore evaluated at T4 and compared as a complete set; the recorded tier is the minimum predicted to decide that scenario. The priority-ordered first verdict is retained only as an operational interface. Detection rate (DR) counts a non-consistent or undecidable verdict before the 180-s timeout. Exact-set accuracy (ESA) requires equality of the complete observed and expected sets, conditional on detection; we also report per-component precision/recall and Hamming loss.

D. Detection, Classification, and Cadence

At the default 0.5-s cadence, all 180 injections were detected and all detected vectors exactly matched their expected class sets (Table IV). The separation of clocks changes the interpretation materially. For S6, median actuation was 10.05 s, median DDL 0.54 s, and median end-to-end latency 10.50 s. The former one-pass value around 7 s was therefore not detector latency. Conversely, S3 had median actuation 0.13 s but median

DDL 9.68 s (P95 10.38 s), exposing the background-policy path.

Across all cadences, 538 of 540 observations produced a verdict, and all 538 detected vectors exactly matched their expected sets. Both misses were S1 in the same repetition: the 0.5-s evaluator observed the manual mutation, but Flux reconciled it before the first 2-s and 10-s polls. This is not discarded as an outlier. It demonstrates the cadence limit predicted by **LH2**: polling a transient relation can miss it even when exact-set identification is perfect when observed.

At 0.5 s, detection was 180/180 (Wilson 95% CI 97.91–100%); at both 2 and 10 s it was 179/180 (96.92–99.90%). Resampling complete three-cadence injection clusters gives 99.63% aggregate detection with a 95% percentile-bootstrap interval of 98.89–100% (50,000 resamples, seed 20260807). These intervals quantify repeated behavior in this protocol; they are not confidence intervals for production effectiveness. For a single transient of duration τ sampled with uniformly random phase at cadence c , the idealized miss probability is $\max(0, 1 - \tau/c)$; a production miss percentage additionally requires the field distribution of τ , which this study does not estimate.

Across all 540 observations, exact-set accuracy was 99.63% and Hamming loss was 0.000617. The two missed S1 labels account for all error: configuration recall was 96.7%; precision for every output and recall for policy, authorization, intent, evidence, and environment were 100% (Table VI).

Table V reports the aggregate effect. DR was 100.0% at 0.5 s and 99.4% at both 2 and 10 s. Conditional ESA remained 100.0%. Median DDL rose from 0.47 to 1.44 and 6.26 s as cadence widened; P95 rose from 10.08 to 10.47 and 14.61 s. P95 is dominated partly by S3’s PolicyReport delay rather than cadence alone, so these are distributions of the complete deciding paths on this laboratory, not calibrated evaluator-service times.

E. Benign Live-Stack Controls

The positive scenarios do not establish specificity. We therefore ran 20 10.5-s drift-free windows (210 s total), balanced round-robin across six controls: satisfied policy revision (C1), rollback accompanied by approval for the new revision (C2), exception removal at expiry (C3), re-tag accompanied by approval for the new digest (C4), autoscaling replica change outside the managed projection (C5), and a legitimate rollout restart (C6). Every window was evaluated at all three cadences.

The controls produced 543 evaluator polls, zero substantive alarms, and zero epistemic warnings (Table VII). This supports **LH3** within the declared projection: the evaluator did not simply classify every change as drift. It does not bound false-positive rates under untested production churn; with zero observed alarms, the defensible statement is 0/543 polls and 0/20 windows, not a claim of zero population probability.

F. Compound Vectors and Extended Benign Soak

The primary campaign establishes each atomic class, but simultaneous changes can become observable at different times. We therefore added a predeclared secondary extension: five repe-

TABLE IV

REPEATED LIVE-STACK RESULTS AT THE DEFAULT 0.5-s CADENCE (20 RUNS PER SCENARIO). EXPECTED IS A CLASS SET; “FIRST VERDICT” IS THE EVALUATOR’S PRIORITY-ORDERED OUTPUT. DR AND ESA ARE PERCENTAGES. ACT., DDL, P95 DDL, END-TO-END (E2E), AND TTE ARE SECONDS; ALL EXCEPT P95 ARE MEDIANS.

ID	Expected set	Observed set	First	DR	ESA	Act.	DDL	P95	E2E	TTE
S1	{configuration}	{configuration}	configuration	100.0	100.0	0.05	0.45	0.65	0.50	0.45
S2	{auth.}	{auth.}	authorization	100.0	100.0	2.34	0.47	0.71	2.90	0.47
S3	{policy}	{policy}	policy	100.0	100.0	0.13	9.68	10.38	9.81	9.68
S4	{auth.}	{auth.}	authorization	100.0	100.0	1.56	0.53	0.68	2.10	0.53
S5	{environment}	{environment}	environment	100.0	100.0	0.03	0.44	0.58	0.46	0.44
S6	{intent, auth.}	{auth., intent}	intent	100.0	100.0	10.05	0.54	0.75	10.50	0.54
S7	{environment}	{environment}	environment	100.0	100.0	0.04	0.45	0.60	0.48	0.45
S8	{auth.}	{auth.}	authorization	100.0	100.0	0.02	0.40	0.67	0.44	0.40
S9	{evidence}	{evidence}	evidence	100.0	100.0	0.01	0.40	0.59	0.41	0.40

TABLE V

CADENCE SENSITIVITY ACROSS NINE SCENARIOS AND 20 REPETITIONS. ESA IS CONDITIONAL ON DETECTION; DDL VALUES ARE SECONDS.

Cadence	Runs	DR (%)	ESA (%)	Median DDL	P95 DDL
0.5	180	100.0	100.0	0.47	10.08
2.0	180	99.4	100.0	1.44	10.47
10.0	180	99.4	100.0	6.26	14.61

TABLE VI

FULL-VECTOR COMPONENT METRICS ACROSS ALL CADENCES. EVIDENCE IS THE EPISTEMIC OUTPUT TRIGGERED BY AN UNDECIDABLE SUBSTANTIVE COMPONENT, NOT A SIXTH SUBSTANTIVE DISTANCE.

Component	TP	FP	FN	Precision	Recall
configuration	58	0	2	100.0%	96.7%
policy	60	0	0	100.0%	100.0%
authorization	240	0	0	100.0%	100.0%
intent	60	0	0	100.0%	100.0%
evidence	60	0	0	100.0%	100.0%
environment	120	0	0	100.0%	100.0%

titions each of live S10 (policy supersession plus expired exception), S11 (artifact substitution plus environment change), and S12 (rollback plus missing live status for continuing authority), observed at the same three cadences. These 15 injections yield 45 correlated cadence observations. After the first non-consistent verdict, the harness continues polling until the complete expected class set appears or the 180-s timeout expires. We call the interval from onset to that event vector completion latency (VCL); DDL remains time to the first honest alert.

All 45 observations detected drift and reached the exact expected final vector (Table VIII). S11 and S12 were complete at the first alert in 15/15 observations each. S10 was complete at the first alert in only 4/15: exception expiry was immediately visible, whereas Kyverno’s asynchronous background scan exposed the policy component later. Its median DDL was 1.33 s, but median VCL was 7.88 s (P95 14.26 s). This is the intended result of retaining a vector over time: an early correct subset

TABLE VII

BENIGN LIVE-STACK CONTROLS. “POLLS” SUMS ALL THREE CADENCES; ALARM WINDOWS COUNTS WINDOWS WITH AT LEAST ONE SUBSTANTIVE DRIFT VERDICT; EPISTEMIC POLLS COUNT HONEST *Undecidable* OUTPUTS SEPARATELY.

ID	Benign control	Windows	Polls	Alarm windows	Epistemic polls
C1	satisfied policy revision	4	108	0	0
C2	approved rollback	4	108	0	0
C3	exception removed at expiry	3	84	0	0
C4	approved artifact re-tag	3	81	0	0
C5	autoscaling replica change	3	81	0	0
C6	legitimate rollout restart	3	81	0	0

must not be mistaken for a complete diagnosis.

The extension then ran one 300-s window for each benign control, 1,800 s in total. The three cadences produced 4,680 evaluator polls, zero substantive alarms, and zero epistemic warnings. The experimental units are the six windows, not the correlated polls; the defensible statement remains the observed 0/6 windows and 0/4,680 polls. The longer soak improves coverage of sustained benign operation but still does not estimate a production false-alarm probability or exercise high-volume concurrent churn.

G. Fault-Contract Checks

The healthy-stream campaign is complemented by 15 deterministic evaluator contract tests. They inject a control-plane/API outage, malformed PolicyReport payload, missing approval basis, missing environment inventory, an uncovered running artifact digest, duplicate valid approval, authenticated retroactive revocation, and one-shot versus continuing authority under missing live status. Five adapter-envelope checks additionally accept a fresh ordered record and reject stale, excessively delayed, reordered, or subject-mismatched evidence. Outage, malformed policy evidence, and missing approval or inventory produce *undecidable* for exactly the affected components; the fresh but uncovered digest produces authorization drift; and duplication is idempotent. These tests verify fail-closed epistemic and transport-boundary semantics, not delay distributions or reliability under a real faulty transport. They are executable with `PYTHONPATH=lab python3 -m`

TABLE VIII

SECONDARY COMPOUND-SCENARIO EXTENSION ACROSS FIVE INJECTIONS AND THREE CADENCES PER SCENARIO. DR AND EXACT REFER TO THE FINAL VECTOR; “FIRST COMPLETE” IS THE FRACTION WHOSE FIRST ALERT ALREADY CONTAINED THAT VECTOR. DDL AND VCL ARE SECONDS.

ID	Expected set	n	DR	Exact	First complete	Median DDL	Median VCL
S10	{policy, auth.}	15	100.0%	100.0%	26.7%	1.33	7.88
S11	{auth., environment}	15	100.0%	100.0%	100.0%	1.25	1.25
S12	{evidence}	15	100.0%	100.0%	100.0%	0.57	0.57

`unittest lab/test_evaluator_contract.py`.

H. Evidence Boundary and Refutation Conditions

The repeated execution strengthens the earlier one-pass realizability check but remains controlled and single-cluster. It uses injected scenarios, file-backed approval and environment streams, a single deployment, no concurrent workload churn during positive injections, and only a 30-minute benign soak. The evaluator degrades unavailable PolicyReport, approval, lineage, and inventory evidence to *undecidable*; however, this campaign does not estimate reliability under real delayed, re-ordered, or lossy transports, API saturation, or stale-cache races; only deterministic adapter contracts cover those temporal cases. It estimates a distribution only for this pinned laboratory protocol. A second reconciler/policy-engine stack and production-backed approval and inventory services remain external-validity work rather than claims smuggled into this execution.

The expected tier/class shape appeared whenever an injection was observed, including every compound extension, supporting **LH1**. The two S1 misses support the cadence-bounded qualification in **LH2** and refute any stronger claim of cadence-independent detection. Zero alarms in the six benign change families support **LH3** within scope. The minimal evidence bundle is constructed synchronously after verdict, so this laboratory does not test a separate, platform-bound evidence retrieval path. Incorrect exact-set output for an observed higher-tier scenario, alarms under these benign controls, or a repeatable class requiring less evidence than Proposition 2 states would refute the corresponding claims.

VIII. METRICS: PROPOSAL AND CRITICAL EVALUATION

We evaluate six candidate metrics against grounding in the model, operationalizability, and incentive safety, committing in advance to rejecting what cannot be justified.

Metric M1 (Governance Drift Distance, GDD): Proposed as: a scalar distance between approved and operational state. Verdict: rejected as primitive; adopted as vector. By Section IV-C, the primitive report is $(GD_{\text{sub}}(t), O(t))$; a scalar GDD is legitimate only as a published-weight composite over the decidable substantive components, with coverage reported separately and the weights understood as policy. It never substitutes for the component-resolved report. (This mirrors the model’s rejection of a scalar distance and is the suite’s structural decision.)

Metric M2 (Drift Detection Latency, DDL): Per class and tier: detection time minus drift onset—in events for closed-world evaluation (measured in Section VI: 0–1 events at the correct tier; 22.6 for naive comparison on S1), in wall-clock for the laboratory. Verdict: adopted. Let t_{inject} be command initiation, t_{onset} the instant the operational relation first becomes inconsistent, t_{detect} the first correct verdict, and t_{evidence} completion of the evidence bundle supporting that verdict. We report actuation latency $t_{\text{onset}} - t_{\text{inject}}$, DDL $t_{\text{detect}} - t_{\text{onset}}$, end-to-end latency $t_{\text{detect}} - t_{\text{inject}}$, and time-to-evidence (TTE) $t_{\text{evidence}} - t_{\text{onset}}$ separately. Onset must be defined per class (e.g., expiry instant for exception drift); the definition ships with the metric. The original one-pass table measured injection-to-verdict and is retained only as a legacy observation in the artifact, not relabeled as DDL.

Metric M3 (Unapproved Change Ratio, UCR): Fraction of observed change events (configuration and environment) not covered by a valid authorization basis at the time of effect. Verdict: adopted, with an evidence caveat: UCR is computable only where approvals are recorded durably; where they are not, UCR’s denominator is measurable but its numerator degrades to undecidable, and the metric must report the undecidable mass separately through O rather than folding it into either pole. With N observed changes, U known unapproved changes, and Q undecidable changes, report the identification interval $UCR_{\text{min}} = U/N$ and $UCR_{\text{max}} = (U + Q)/N$ rather than a fabricated point estimate.

Metric M4 (Policy-Version Divergence, PVD): For each running deployment: the set (and count) of policy controls, in the currently governing version, that the deployment violates while remaining covered only by an older pinned basis—estate-wise, the distribution of “policy age” (versions or time between pinned and current). Verdict: adopted as a set/age pair; rejected as a bare count, since control violations are policy-weighted and a count of one critical control must not compare equal to one cosmetic control.

Metric M5 (Authorization Drift Ratio, ADR): Fraction of authorization dependencies currently relied upon in an estate whose instruments are expired, revoked, or subject-mismatched while still relied upon—S2/S8-type conditions as a standing estate measurement. Verdict: adopted; it is directly computable from T2/T3 evidence and is the metric that makes “exception hygiene” a number.

Metric M6 (Governance Conformance Coverage, GCC): Fraction of running deployments for which governance con-

sistency is evaluable at all—i.e., an admitted basis exists, and every component divergence can be computed from available streams. Verdict: adopted, and placed first in reporting order: coverage bounds the meaning of every other metric (a superb ADR over 20% GCC is a statement about 20% of the estate), and low GCC is itself the actionable finding that approved-state recording is missing.

Reporting discipline: GCC first; then the class-resolved vector metrics (DDL per tier, UCR with undecidable mass, PVD as set/age, ADR); composites last, weights published. The gaming surfaces are the usual ones—weakening policy lowers PVD; widening approvals lowers UCR—and, as in any governance measurement, the counters are meaningful only jointly with the policy and approval baselines they are computed against.

A. What the Present Data Can and Cannot Instantiate

The live campaign identifies DDL directly and exercises GCC semantics, but it does not instantiate estate-level GCC because its units are observations, not running deployments. All 60 S9 cadence observations intentionally fail the approval evidence contract, so observation-level full-vector evaluability is 480/540 (88.9%) in the scenario-balanced positive set and 543/543 polls in the benign controls. This is a contract exercise, not an estate estimate. UCR, PVD, and ADR are not reported from these injections: their denominators are deployment populations or naturally occurring change and authorization populations, whereas this case-control design fixes scenario counts by construction. Relabeling those fractions as operational rates would manufacture prevalence.

For an estate, aggregate only after reporting GCC: compute each metric over evaluable deployments, retain unevaluable mass separately, and stratify by service criticality, environment, and evidence tier. A fleet-wide scalar may then be a published-weight summary, but it must not erase class counts or let a high-coverage low-risk stratum conceal an unevaluable critical stratum.

IX. RELATED WORK

A. The Missing Join

The closest systems do observe many of the ingredients of governance drift, but they organize them around plane-local questions: desired versus observed configuration, state versus current policy, artifact versus admission rule, or environment versus benchmark. Table IX makes the gap explicit. The table compares each line of work by its *native decision object*, not by every integration a commercial product might expose. A triangle means that the line carries a useful slice but does not natively join that slice to the time-indexed admitted basis.

This fragmentation explains why the aggregate relation remained easy to miss: a deployment can be green in each available local view while the join among present state, current context, and the recorded admitted basis is broken. The contribution is not the claim that no existing mechanism can be composed to cover several columns. It is the explicit temporal

three-way relation, the class-resolved verdicts it yields, and the evidence coverage required to decide it.

B. Configuration Drift and Desired-State Systems

Configuration drift detection is mature practice: reconciler status in GitOps [3, 4, 1, 2], cloud configuration recorders [6], IaC drift tools [7], and the configuration-management control tradition [14, 15] that institutionalizes baseline comparison. IaC research studies the quality and smells of the configuration artifacts themselves [28, 29, 30]. All of it operates in the configuration vocabulary and against the *current* desired state; Proposition 1 bounds what that vocabulary can decide, and Finding F4 measures the bound. Our claim is containment: this literature solves one class of six, and solves it well—T0n in our architecture is this literature.

C. Policy-as-Code, Admission, and Continuous Compliance

Policy engines [9, 10] and admission control [8] evaluate governance predicates at a point in time; posture management [13] and hardening baselines [27] scan environments against benchmark policy; continuous-validation features in attestation-gated deployment [11] re-check the artifact-policy slice after admission. These supply, respectively, our T1 evaluation semantics, part of T4’s observation, and a one-tier precedent for continuous re-evaluation. Commercial continuous-compliance and CSPM products may integrate several of these signals, but their native decision object remains current state versus current rule or benchmark; this is not an exhaustive feature audit. What the documented mechanisms do not provide is the *approved-state reference*: they compare state against current policy, not against the basis under which the state was admitted, so they cannot distinguish “was never compliant” from “was admitted legitimately and the world moved”—the distinction carrying the remediation semantics of Section IV-C, and the defining relation of governance drift. Runtime compliance-monitoring research systematically studies rule evaluation over process-event streams and fine-grained violation feedback [31]. That literature supplies monitoring languages and execution semantics; its native reference is a compliance rule or process model, not a deployment-specific immutable admission event. The distinction is complementary rather than competitive: such monitors can implement T1, while $B_x(t)$ determines why the same current violation is classified as post-admission drift rather than absence of an admitted basis.

D. Supply-Chain Integrity and Authorization Evidence

Attestation frameworks bind artifacts to build and review facts [19, 20, 23]; they are T3’s substrate and make S4/S8-class drift decidable. Their orientation is admission-time verification; governance drift adds the post-admission temporal dimension (validity windows, revocation, supersession) and the join against G_{app} . Records-side, durable approval evidence and transparency services [32, 33, 24] determine whether the evaluator’s verdicts are decidable at all—our observability mask and GCC metric make that dependence explicit rather than assumed.

TABLE IX
REPRESENTATIVE NATIVE DECISION SCOPE OF ADJACENT WORK. ✓ = NATIVE DECISION OBJECT; △ = PARTIAL INPUT OR SLICE; – = NOT NATIVE.
THE COMPARISON IS NOT AN EXHAUSTIVE PRODUCT FEATURE AUDIT.

Line of work	Config.	Policy	Auth.	Intent	Evidence	Environ.	Joins admitted basis
Desired-state/GitOps reconciliation [3, 4, 1]	✓	–	–	–	–	–	–
IaC drift/configuration recorders [6, 7]	✓	–	–	–	–	△	–
Policy-as-code/background scans [9, 10]	–	✓	–	–	–	–	–
Posture and hardening scans [13, 27]	△	✓	–	–	–	✓	–
Supply-chain attestations [19, 23]	–	△	△	△	✓	–	–
Continuous artifact validation [11]	–	✓	✓	–	△	–	–
Governance drift (this work)	✓	✓	✓	✓	✓	✓	✓

E. Access Control, Runtime Verification, and Drift Theory

Temporal validity of authorization has deep roots in temporal authorization and role models [34, 35], while usage control adds ongoing decisions, mutable attributes, obligations, and conditions [36, 37]. These works justify the one-shot/continuing distinction rather than leaving validity as a Boolean. Classical access-control theory [17, 38] and zero-trust’s continuous-evaluation stance [39] provide the broader foundation; governance drift applies it to deployment admission history. Runtime verification [40, 41] monitors executions against temporal specifications; our evaluator is kin—governance consistency is a safety-style property over joined state and context streams—but the specification here is not a given formula: it is reconstructed per deployment from the approved snapshot, which is why recording G_{app} is the architecture’s prerequisite rather than an optimization. Concept drift [21] shares only the word: it names distribution change in data streams, not divergence from an approval basis; we flag it to prevent terminological import of its detectors, which answer a statistical question ours does not ask. Industry usage of “governance” alongside drift management [22] treats governance as the program that suppresses configuration drift; we found no documented formal treatment of drift of the governance relation itself, and a term search (queries “governance drift” alone and with `cloud`, `infrastructure`, `configuration`; ACM DL, IEEE Xplore, general scholarly indices; August 2026) found the compound used in internet-governance and organizational senses unrelated to this object. We therefore narrow the novelty claim to what this review supports: we found no documented work that jointly formalizes current operational state, evolving governance context, and an immutable admitted basis as a class-resolved post-admission relation.

F. Novelty, Answered Directly

Is this just configuration drift? No—with the argument’s weight placed where it belongs. The load-bearing evidence is analytic: constructed, tool-realizable counterexamples in which configuration equality and governance inconsistency coexist (Figure 3), and the dependency result that no configuration-pair detector decides the property (Proposition 1). The executed staircase (Table II) supports these as an implementation-validation and noise study, not as independent field evidence. And we concede the mechanical point a skeptic will press: every governance-tier detector is itself a

comparison—against a recorded basis, a validity clock, or a coverage set. The claim is not a new mechanism but a different *relation*: three-place (state, current context, admitted basis) where configuration drift is two-place and memoryless, with the consequences Section X enumerates. *Is it policy compliance scanning?* Scanning evaluates current state against current policy; governance drift is three-way—state, current context, and *approved basis*—and the third term changes both the verdicts (legitimately-admitted-since-superseded $\neq$ never-compliant) and the remediations. *Is it continuous compliance scanning or continuous validation?* Per tier, largely yes—and we credit it: Kyverno-style background scanning continuously re-evaluates resources against current policy (T1); configuration recorders and posture management watch unmanaged resources (much of T4); continuous validation re-checks artifact policy (part of T3) [10, 6, 13, 11]. Several of our scenarios trigger *some* existing detector. We found no documented mechanism that provides the complete join against the admitted basis with mode-aware validity and class-resolved verdicts—the increment that turns “a scanner fired” into “this running state is no longer covered by what admitted it, for this reason, with this remediation family.” That increment is architectural and, we argue, decision-relevant; quantifying its operational value on real estates remains future field work. The present laboratory tests only the three hypotheses stated in Section VII.

X. DISCUSSION AND LIMITATIONS

A. Legitimacy, Blame, and the Policy-Drift Question

Measuring divergence is not assigning fault. Policy drift in particular is usually the *organization’s own* motion; flagging a deployment as governance-inconsistent under π_8 says migration or re-approval is due, not that anyone erred. We consider this a feature of the vector semantics: because classes are separated, an estate can adopt distinct dispositions per class (auto-reconcile configuration drift; ticket policy drift with a migration SLA; page on authorization drift; block on intent drift)—consistent with the evidence that indiscriminate heavyweight gating harms delivery without improving stability [42, 43]—turning the taxonomy into an operating policy rather than an alarm firehose. What the model deliberately does not decide—and no measurement can—is the disposition itself. Table I makes the distinction systematic: class selects a primary risk channel and response family, while blast radius, duration, reversibility, consequence, and evidence coverage determine ac-

tual severity. The vector must not be collapsed into a universal danger ranking.

B. Evidence Failure and Business Availability

An *undecidable* verdict is epistemic, not an automatic shut-down command. The safe default is fail-closed for the *governance claim*—the system must not report conformance without evidence—while availability policy is chosen separately. A new admission may be blocked when T1–T3 evidence is missing; an already-running critical service may remain available under a bounded grace period, elevated alert, and evidence-repair SLO. Conversely, a retroactive revocation or known uncovered digest is a substantive verdict, not evidence degradation, and may justify immediate containment. The component, service criticality, blast radius, and maximum tolerated evidence age must therefore select fail-open or fail-closed action in a published policy. This preserves business availability without laundering uncertainty into a false “consistent” state.

C. Limitations

The approved-state prerequisite. Everything downstream of Section V assumes an admitted basis $B_x(t)$ was recorded durably; where it was not, verdicts degrade to undecidable, and the honest headline is GCC, not drift rates. This is a real adoption constraint, not a footnote: estates without durable approval evidence cannot measure their governance drift, and discovering that is arguably the measurement’s first value. *Bounded live empirics.* The laboratory executed 20 repetitions of each scenario at three evaluator cadences on one Kind/Flux/Kyverno stack, plus 20 benign control windows. Its 538/540 detection observations, 538/538 conditional exact-set agreements, zero conditional Hamming loss, and zero substantive alarms or epistemic warnings in 543 benign-control polls are supplemented by 45/45 exact compound-vector completions and a 30-minute, 4,680-poll benign soak with no substantive or epistemic alarm. They establish bounded realizability and within-laboratory behavior. They do not establish natural occurrence, field prevalence, production reliability, universal false-alarm rates, stream-loss tolerance, or transfer across organizations. The two cadence misses are retained because they expose a real observability boundary for transient drift. *Single-deployment granularity.* The model is per-deployment; estate-level aggregation (fleet drift, correlated policy supersessions) raises weighting questions we defer to the metrics’ published-composite discipline. *Environment scope.* d_{env} is bounded by what σ_B declares; assumptions never written down cannot drift detectably—an instance of the general dependence of governance measurement on capture discipline. *Remediation dynamics.* We model detection, not the control loop that acts on it; closing the loop (auto-re-approval workflows, drift-driven migration) is future work with its own stability questions.

D. Outlook

The machinery this work presupposes—durable approval evidence, and an account of how such evidence decays—and the drift semantics defined here compose naturally: approved-state

snapshots are exactly the artifacts durable approval linkage produces, and evidence drift is evidence debt manifesting inside the drift vector. We have kept this manuscript self-contained, but the research program is one: make the relationship between what organizations approve and what their systems do continuously demonstrable—at admission, over time, and after the fact.

XI. DATA AND CODE AVAILABILITY

The manuscript source, reference implementation, closed-world data, independent no-join baseline, repeated and compound live-stack observations, benign-control records, and generated tables are archived as version 1.5.0 at <https://doi.org/10.5281/zenodo.21845707>; the version-independent concept DOI is <https://doi.org/10.5281/zenodo.21841458>. Development history and the tagged release are at <https://github.com/obedebessa/governance-drift> (tag v1.5.0). The archive’s RELEASE_MANIFEST.md records SHA-256 digests for the manuscript PDF and source package, while the Git tag binds the released tree. Version 1.2.0 remains available as the predecessor at <https://doi.org/10.5281/zenodo.21843860>.

The dependency-free closed-world study is regenerated with `python3 code/detector_study.py`; `PYTHONPATH=lab python3 -m unittest -v lab/test_evaluator_contract.py` checks fail-closed evidence semantics; `python3 scripts/analyze_lab_uncertainty.py` reproduces the Wilson and injection-cluster bootstrap intervals; `python3 scripts/analyze_lab_extension.py` validates and summarizes the compound-vector extension and extended benign soak; `python3 scripts/verify_artifact.py` checks its canonical outputs and the frozen laboratory results. With Docker, Kind, kubectl, Git, and the pinned images available, `lab/bootstrap.sh` creates the cluster and `python3 lab/run_repeated_experiment.py` executes the seeded 20-by-9 protocol and benign controls. Finally, `python3 scripts/verify_lab_results.py` verifies the raw CSV/JSON observations and the generated summary tables. Platform versions, seed, cadences, raw timestamps, timeout outcomes, expected and observed class sets, priority projections, component metrics, Hamming loss, and evidence-completion times are retained in `lab/results/`; the secondary campaign is in `lab/results_extension/`.

XII. CONCLUSION

Reconciliation solved configuration drift so well that its green status has come to stand for a property it does not check. This paper named the difference: governance drift, the divergence of operational reality from the state, policy context, authorization basis, intent, and assumptions under which it was approved. We separated five substantive drift components and a transversal epistemic evidence condition formally; estab-

lished, by constructed counterexample and by a definitional-dependency argument, that configuration equality does not imply governance consistency and that no configuration-pair detector can decide it; modeled governance divergence as an irreducibly component-wise substantive vector plus observability mask, anchored to a recorded approval event; organized detection as cumulative evidence tiers; and checked, in an executed and fully reproducible closed-world study with class-set ground truth and benign governance churn in its control, that a definition-faithful reference implementation realizes the model’s detectability staircase: configuration comparison decides exactly one of twelve drift scenarios and each governance tier detects precisely the components whose deciding facts it carries, earning zero false alarms against governance noise. RQ0 is thereby answered with its conditions stated: governance drift is formally representable (the vector GD), measurable exactly where the admitted basis was recorded (coverage, GCC, bounds everything else), and detectable to the depth of the evidence tiers an estate maintains. The repeated Kubernetes/GitOps laboratory executed 20 repetitions of all nine scenarios at 0.5-, 2-, and 10-s cadences on a pinned Kind/Flux/Kyverno stack. It detected 538 of 540 observations; all 538 detected outputs exactly matched the expected sets, yielding zero conditional Hamming loss. Twenty benign windows covering six change families produced zero substantive alarms and zero epistemic warnings in 543 polls. Separate injection, onset, detection, and evidence clocks show that actuation and DDL are distinct: for example, S6’s median actuation was 10.05 s while median DDL was 0.54 s at the default cadence. These results establish bounded realizability and within-laboratory behavior, not natural occurrence, prevalence, production reliability, or universal latency distributions. An independent no-join composite diagnoses only 9/12 closed-world scenario sets versus 12/12 for the history-aware join, isolating the increment from alarm volume. The live compound extension reaches exact final vectors in 45/45 cadence observations and exposes why first alert and vector completion are distinct: S10’s median DDL is 1.33 s but its median complete diagnosis is 7.88 s. A 30-minute benign soak adds 4,680 polls without substantive or epistemic alarms; it strengthens the bounded specificity check without being misrepresented as a production-rate estimate.

The definitions are offered for reuse; the matrix is offered as a template for evaluating any governance-drift detector; and the central lesson is offered to practice in one sentence: a system can match its manifest perfectly and still be running outside everything that made it legitimate—and whether an organization can *see* that is an architectural choice, made tier by tier, that this paper makes explicit.

REFERENCES

- [1] OpenGitOps Project, “GitOps principles, version 1.0.0.” <https://opengitops.dev/>, 2021. CNCF. Accessed August 2026.
- [2] F. Beetz and S. Harrer, “GitOps: The evolution of DevOps?,” *IEEE Software*, vol. 39, no. 4, pp. 70–75, 2022.
- [3] Argo Project, “Argo CD: Declarative GitOps continuous delivery for Kubernetes.” <https://argo-cd.readthedocs.io/>, 2026. Accessed August 2026.
- [4] Flux Project, “Flux: The GitOps family of projects.” <https://fluxcd.io/>, 2026. Accessed August 2026.
- [5] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and Kubernetes,” *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.
- [6] Amazon Web Services, “AWS Config developer guide: Detecting drift and evaluating resources.” <https://docs.aws.amazon.com/config/latest/developerguide/>, 2026. Accessed August 2026.
- [7] Snyk, “driftctl: Detect, track and alert on infrastructure drift.” <https://docs.driftctl.com/>, 2023. Accessed August 2026.
- [8] Kubernetes Project, “Admission controllers reference; validating admission policy.” <https://kubernetes.io/docs/reference/access-authn-authz/>, 2026. Accessed August 2026.
- [9] Open Policy Agent Project, “Open policy agent documentation.” <https://www.openpolicyagent.org/docs/>, 2026. CNCF. Accessed August 2026.
- [10] Kyverno Project, “Kyverno: Kubernetes native policy management.” <https://kyverno.io/docs/>, 2026. Accessed August 2026.
- [11] Google Cloud, “Binary authorization documentation, including continuous validation.” <https://cloud.google.com/binary-authorization/docs>, 2026. Accessed August 2026.
- [12] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O’Reilly Media, 2016.
- [13] Microsoft, “Cloud security posture management (CSPM) documentation, microsoft defender for cloud.” <https://learn.microsoft.com/en-us/azure/defender-for-cloud/concept-cloud-security-posture-management>, 2026. Accessed August 2026.
- [14] A. Johnson, K. Dempsey, R. Ross, S. Gupta, and D. Bailey, “Guide for security-focused configuration management of information systems,” Tech. Rep. NIST Special Publication 800-128, National Institute of Standards and Technology, 2011.
- [15] Joint Task Force, “Security and privacy controls for information systems and organizations,” Tech. Rep. NIST Special Publication 800-53, Revision 5, National Institute of Standards and Technology, 2020.

- [16] United States Congress, “Sarbanes–oxley act of 2002.” Public Law 107–204, 116 Stat. 745, 2002.
- [17] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [18] AXELOS, “ITIL foundation: ITIL 4 edition.” TSO (The Stationery Office), 2019.
- [19] OpenSSF, “SLSA specification, version 1.2.” <https://slsa.dev/spec/v1.2/>, 2025. Accessed August 2026.
- [20] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappel, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in *Proceedings of the 28th USENIX Security Symposium*, pp. 1393–1410, USENIX Association, 2019.
- [21] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.
- [22] ControlMonkey, “Cloud governance best practices: Five ways to prevent drift.” <https://controlmonkey.io/blog/cloud-governance-best-practices-devops/>, 2025. Representative industry usage conflating drift management with governance; accessed August 2026.
- [23] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software signing for everybody,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 2353–2367, ACM, 2022.
- [24] Amazon Web Services, “AWS CloudTrail user guide.” <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/>, 2026. Accessed August 2026.
- [25] OpenTelemetry Project, “The opentelemetry specification.” <https://opentelemetry.io/docs/specs/otel/>, 2026. CNCF. Accessed August 2026.
- [26] Kubernetes SIGs, “kind: Kubernetes in Docker.” <https://kind.sigs.k8s.io/>, 2026. Accessed August 2026.
- [27] National Security Agency and Cybersecurity and Infrastructure Security Agency, “Kubernetes hardening guide, version 1.2.” Cybersecurity Technical Report, 2022.
- [28] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, “A systematic mapping study of infrastructure as code research,” *Information and Software Technology*, vol. 108, pp. 65–77, 2019.
- [29] A. Rahman, C. Parnin, and L. Williams, “The seven sins: Security smells in infrastructure as code scripts,” in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pp. 164–175, IEEE, 2019.
- [30] T. Sharma, M. Fragkoulis, and D. Spinellis, “Does your configuration code smell?,” in *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*, pp. 189–200, ACM, 2016.
- [31] L. T. Ly, F. M. Maggi, M. Montali, S. Rinderle-Ma, and W. M. P. van der Aalst, “Compliance monitoring in business processes: Functionalities, application, and tool-support,” *Information Systems*, vol. 54, pp. 209–234, 2015.
- [32] H. Birkholz, A. Delignat-Lavaud, C. Fournet, Y. Deshpande, and S. Lasker, “An architecture for trustworthy and transparent digital supply chains (SCITT).” Internet-Draft draft-ietf-scitt-architecture-22, IETF, 2025. Work in Progress, October 2025.
- [33] L. Moreau and P. Missier, “PROV-DM: The PROV data model.” W3C Recommendation, 2013. <https://www.w3.org/TR/prov-dm/>. April 30, 2013.
- [34] E. Bertino, C. Bettini, and P. Samarati, “A temporal authorization model,” in *Proceedings of the 2nd ACM Conference on Computer and Communications Security*, pp. 126–135, 1994.
- [35] E. Bertino, P. A. Bonatti, and E. Ferrari, “TRBAC: A temporal role-based access control model,” *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 191–233, 2001.
- [36] J. Park and R. Sandhu, “The UCONABC usage control model,” *ACM Transactions on Information and System Security*, vol. 7, no. 1, pp. 128–174, 2004.
- [37] U. Schöpp, C. Xu, A. Ibrahim, F. Faghih, and T. Dimitrakos, “Specifying a usage control system,” in *Proceedings of the 28th ACM Symposium on Access Control Models and Technologies*, 2023.
- [38] D. D. Clark and D. R. Wilson, “A comparison of commercial and military computer security policies,” in *Proceedings of the 1987 IEEE Symposium on Security and Privacy*, pp. 184–194, IEEE, 1987.
- [39] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” Tech. Rep. NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.
- [40] M. Leucker and C. Schallhart, “A brief account of runtime verification,” *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.

- [41] A. Pnueli, “The temporal logic of programs,” in *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 46–57, IEEE, 1977.
- [42] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. Portland, OR, USA: IT Revolution Press, 2018.
- [43] DevOps Research and Assessment (DORA), “Accelerate: State of DevOps 2019.” Google Cloud, 2019. <https://dora.dev/research/2019/>. Accessed August 2026.